

```
import tensorflow as tf
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.datasets import mnist
from tensorflow.keras.optimizers import Adam
```

```
print("Available GPUs:", tf.config.list_physical_devices('GPU'))
```

```
Available GPUs: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 2s 0us/step
```

```
import numpy as np
```

```
learning_rate_range = [1e-4, 1e-2]
hidden_units_options = [64, 128, 256, 512]
batch_sizes_options = [16, 32, 64, 128]
```

```
num_random_samples = 10
```

```
best_accuracy = 0
```

```
best_params = {}
```

```
with tf.device('/GPU:0'):
```

```
    for i in range(num_random_samples):
```

```
        lr = np.exp(np.random.uniform(np.log(learning_rate_range[0]), np.log(learning_rate_range[1])))
```

```
        units = np.random.choice(hidden_units_options)
```

```
        batch = np.random.choice(batch_sizes_options)
```

```
        model = Sequential([
            Flatten(input_shape=(28, 28)),
            Dense(units, activation='relu'),
            Dense(10, activation='softmax')
        ])
```

```
        model.compile(
            optimizer=Adam(learning_rate=lr),
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy']
        )
```

```
        model.fit(
            X_train, y_train,
            epochs=5,
            batch_size=batch,
            validation_split=0.2,
            verbose=0
        )
```

```
        _, acc = model.evaluate(X_test, y_test, verbose=0)
```

```
        print(f"Random Sample {i+1}: LR={lr:.6f}, Units={units}, Batch={batch} → Accuracy={acc:.4f}")
```

```
        if acc > best_accuracy:
```

```
            best_accuracy = acc
```

```
            best_params = {
```

```
                'learning_rate': lr,
```

```
                'units': units,
```

```
                'batch_size': batch
```

```
            }
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/
super().__init__(**kwargs)
```

```
Random Sample 1: LR=0.001354, Units=128, Batch=128 → Accuracy=0.9737
```

```
Random Sample 2: LR=0.001373, Units=128, Batch=128 → Accuracy=0.9705
```

```
Random Sample 3: LR=0.001480, Units=512, Batch=16 → Accuracy=0.9801
```

```
Random Sample 4: LR=0.006340, Units=512, Batch=32 → Accuracy=0.9653
```

```
Random Sample 5: LR=0.000101, Units=128, Batch=64 → Accuracy=0.9384
```

```
Random Sample 6: LR=0.001244, Units=128, Batch=128 → Accuracy=0.9737
```

```
Random Sample 7: LR=0.003215, Units=256, Batch=64 → Accuracy=0.9753
```

```
Random Sample 8: LR=0.006782, Units=64, Batch=16 → Accuracy=0.9606
Random Sample 9: LR=0.000591, Units=128, Batch=64 → Accuracy=0.9727
Random Sample 10: LR=0.001941, Units=512, Batch=32 → Accuracy=0.9721
```

```
print("\nBest Accuracy from random Search:", best_accuracy)
print("Best Hyperparameters:", best_params)
```

```
Best Accuracy from random Search: 0.980999760627747
Best Hyperparameters: {'learning_rate': np.float64(0.0014800636737210794), 'units': np.int64(512), 'batch_size': np.int64(16)}
```

```
final_model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(best_params['units'], activation='relu'),
    Dense(10, activation='softmax')
])

final_model.compile(
    optimizer=Adam(learning_rate=best_params['learning_rate']),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = final_model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=best_params['batch_size'],
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 1/10
3000/3000 ————— 11s 3ms/step - accuracy: 0.9031 - loss: 0.3167 - val_accuracy: 0.9653 - val_loss: 0.1182
Epoch 2/10
3000/3000 ————— 9s 3ms/step - accuracy: 0.9724 - loss: 0.0867 - val_accuracy: 0.9697 - val_loss: 0.1008
Epoch 3/10
3000/3000 ————— 8s 3ms/step - accuracy: 0.9837 - loss: 0.0522 - val_accuracy: 0.9713 - val_loss: 0.0995
Epoch 4/10
3000/3000 ————— 9s 3ms/step - accuracy: 0.9860 - loss: 0.0415 - val_accuracy: 0.9712 - val_loss: 0.1062
Epoch 5/10
3000/3000 ————— 9s 3ms/step - accuracy: 0.9889 - loss: 0.0355 - val_accuracy: 0.9741 - val_loss: 0.1091
Epoch 6/10
3000/3000 ————— 8s 3ms/step - accuracy: 0.9918 - loss: 0.0271 - val_accuracy: 0.9755 - val_loss: 0.1098
Epoch 7/10
3000/3000 ————— 9s 3ms/step - accuracy: 0.9917 - loss: 0.0249 - val_accuracy: 0.9772 - val_loss: 0.1157
Epoch 8/10
3000/3000 ————— 9s 3ms/step - accuracy: 0.9945 - loss: 0.0164 - val_accuracy: 0.9767 - val_loss: 0.1440
Epoch 9/10
3000/3000 ————— 9s 3ms/step - accuracy: 0.9937 - loss: 0.0207 - val_accuracy: 0.9758 - val_loss: 0.1580
Epoch 10/10
3000/3000 ————— 8s 3ms/step - accuracy: 0.9941 - loss: 0.0194 - val_accuracy: 0.9741 - val_loss: 0.1622
```

```
loss, accuracy = final_model.evaluate(X_test, y_test, verbose=0)
print(f"\nFinal Test Accuracy using Best Hyperparameters: {accuracy*100:.2f}%")
```

```
Final Test Accuracy using Best Hyperparameters: 97.54%
```

Start coding or [generate](#) with AI.

